

Agentic AI for Coding

A hands-on introduction to Claude Code

Fabian Zills · Tobias Fellner · Institute for Computational Physics (ICP), University of Stuttgart · 2026

The promise

By the end you will have **shipped a small, tested, reviewed pull request** with Claude Code — using a workflow you can reuse on your own research code.

- Assumes: **Python + git**. No prior Claude Code needed.
- Style: short input, **lots** of hands-on.
- Practice repo: `odelab`, a tiny 1D ODE integrator.

Agenda

#	Section
0–1	The agent loop + a live demo
2	<code>uv</code> crash course
3	Understand the repo
4	Steering the agent
	Break
5	Ship a feature: plan → TDD → review → PR
6–7	Pitfalls · advanced demo · wrap

§0

The agent loop

What makes it "agentic"?

Not autocomplete. Not chat. The model **acts in a loop** with tools:

goal → **plan** → **act** → **observe** → **revise**

- **act** — read/write files, run commands, run tests
- **observe** — read the output, errors, test results
- **revise** — adjust, and go again — until the goal is met

Vibe-coding vs. AI-assisted engineering

Vibe-coding	Disciplined use
Prompt, accept, hope	Plan, verify, review
You read little	You own architecture & review
Fine for throwaway	Right for real work

*Studies of professional developers: they **plan before implementing and validate every output.***

When to delegate

Agents shine on **small, well-specified** tasks.

They struggle where **architectural judgement** is needed.

-  "Add an RK4 step and test its convergence order"
-  "Redesign how the solver handles stiff systems"

Today's task is deliberately small — and that's the point.

§1

A live demo

Watch the loop (live)

Plain script → a proper CLI, in real time.

`scripts/run_decay.py` → add `argparse`, `--help`, type hints

Watch for:

- the **plan** before any code
- a **mistake** and the **recovery** — that's the real skill

§2

uv crash course

Why `uv`?

One fast tool for **environments + packages + running**.

- Reproducible: a locked `uv.lock` everyone shares
- No manual `venv` juggling
- `uv run` = "run this inside the project's environment"

Optional niceties: `direnv` (auto-activate) · `UV_CACHE_DIR` (shared cache).

The four commands you need

```
uv sync          # create/refresh the env from pyproject + lock
uv run pytest    # run something inside that env
uv run python scripts/run_decay.py
uv add matplotlib # add a dependency
```

Your turn — get the repo running

```
uv sync
uv run pytest                # 4 tests should pass (green baseline)
uv run python scripts/run_decay.py
uv add matplotlib            # you'll plot later
```

✓ **Done when:** tests are green and the decay script prints a result.

§3

Understand the repo first

Explore before you change

The **Explore** phase: build a mental map before touching code.

- **Plan mode** = read-only. Claude reads and answers; it doesn't edit.
- Ask it to map structure, entry points, and tests.
- Understanding now is cheaper than unwinding a wrong change later.

Your turn — ask Claude to explain `ode1ab`

In **plan mode**, ask Claude to:

| *explain this repository — structure, what each module does, how to run it and the tests*

then follow up:

| *where would I add a new integration method, and what should I test?*

✓ **Done when:** you can describe the repo in a sentence and know where RK4 will go.

§4

Steering the agent

CLAUDE.md — project memory

Instructions Claude reads **every session**: conventions, how to run things, gotchas.

Keep it concise. For each line ask: *"would removing this cause a mistake?"* If not — cut it.

*Bloated CLAUDE.md files get **ignored** — the important rules drown in the noise.*

More levers

- **Skills** — reusable instructions you invoke on demand; use existing ones or write your own (e.g. a personal docstring style).
- **Model & reasoning effort** — more power for hard, ambiguous tasks; smaller/faster for routine edits. Not "biggest by default."
- **Permissions & safety** — plan mode vs. accept-edits; `/permissions` . Keep the agent on a leash.

Your turn — feel the steering

Pick **one or two**:

1. Add a rule to `CLAUDE.md` (e.g. *"use NumPy-style docstrings"* or *"prefer dataclasses"*), ask Claude to write a small helper, then **toggle the rule** and regenerate — spot the difference.
2. Use, or write, a tiny **skill** encoding a personal convention.
3. Compare **plan mode** vs **accept-edits** on a trivial edit; open `/permissions`.

✅ **Done when:** you've seen config visibly change Claude's output.



Break

§5

Ship a feature

plan → TDD → review → PR · the main event

The task: add RK4

Add a **4th-order Runge–Kutta** step alongside Euler — and prove it's better.

The disciplined loop:

plan → **test (red)** → **implement (green)** → **review** → **PR**

Canonical Claude workflow is Explore → Plan → Code → Commit.

*The **TDD** and **review** steps are our deliberate additions.*

Phase 1 — Plan

Use **plan mode** / brainstorming. Get a plan *before* any code.

- Have Claude draft the plan
- **Read it. Annotate it.** Then: *"address all notes, don't implement yet."*
- Don't let it code until the plan is right.

Your turn — plan RK4

Ask Claude (plan mode) to plan adding an `rk4_step` to `ode lab`, reusing the existing `integrate(..., method=...)` hook.

- Review the plan; add a note or two.
- Tell it to revise the plan — **without coding yet**.

✓ **Done when:** you have a plan you'd sign off on.

Phase 2 — Test first (red)

Give the agent a **pass/fail check**. For numerical code, that's a test.

- RK4 matches the analytic decay to a tight tolerance
- **Convergence order ≈ 4** : double the steps $\rightarrow$ error drops **$\sim 16\times$**

Your turn — write the failing test

*Ask Claude to add tests for `rk4_step` **before** implementing it — matching the analytic solution and checking convergence order.*

```
uv run pytest -q          # should be RED — rk4_step doesn't exist yet
```

✓ **Done when:** a new test exists and fails for the right reason.

Phase 3 — Implement (green)

Now let Claude implement — and **let the tests close the loop**.

```
uv run pytest -q      # iterate until green
```

Without a runnable check, "*looks done*" is the only signal — and **you** become the verification loop.

Your turn — make it green

| Ask Claude to implement `rk4_step` until your tests pass.

```
uv run pytest -q
```

✓ **Done when:** all tests pass, including your convergence-order test.

Phase 4 — Review

Two reviewers: the agent, and **you**.

- `/code-review`, or ask Claude to **adversarially** review its own diff
- **Read the diff yourself** — correctness, clarity
- Request changes; re-run the tests



Your turn — review the diff

```
git diff
```

Ask Claude to review its own change for correctness & clarity — then read it yourself and request one improvement.

✓ **Done when:** you've read every changed line and tests still pass.

Phase 5 — Small, targeted PR

Commit in small steps; open a **small** PR.

```
git switch -c add-rk4
git add -A
git commit -m "Add RK4 integrator + convergence tests"
git push -u origin add-rk4
```

You won't meaningfully review a 1000-line PR. Keep it small.

Your turn — open the PR

Create a branch, commit, push, and open a PR describing **what & why** and **how you tested it**.

✅ **Done when:** your PR is open and CI is running the tests.

Fast finishers: the plots/examples misbehave — hunt & fix the planted **bugs**
(`scripts/plot_decay.py` and the logistic example).

§6

Common failure patterns

Five ways it goes wrong — and the fix

1. **Kitchen-sink session** → `/clear` between unrelated tasks
2. **Correcting over & over** → after 2 tries, `/clear` + a better prompt
3. **Over-specified** `CLAUDE.md` → prune ruthlessly
4. **Trust-then-verify gap** → always give a check; can't verify → don't ship
5. **Infinite exploration** → scope narrowly, or use a subagent

§7

Beyond the basics

A glimpse further (live)

One quick live demo — pick one:

- **Subagents** — delegate a scoped task to a fresh context
- **Hooks** — deterministic automation on events (format, test, block)
- **MCP / memory** — connect external tools & persistent context

Take it further

Teasers: GitHub Actions auto-review + human review · git worktrees for parallel work

For your research code:

- numerical / property tests = your verifiable check
- `uv` = reproducible environments

Resources: Anthropic Claude Code docs · `research_agentic_coding_workshops.md` (in this repo)

goal → plan → act → observe → revise

Plan first · verify everything · review before you ship · keep it small.

Now go run the loop on your own code.